Kano Model Analyzer: Requirements Document

Course: Software Construction and Development

Instructor: Engr. Muhammad Umer Haroon

Version: v0.1.0 (initial requirements, subject to change)

1. Project Title and Group Members

Project: Kano Model Analyzer

#	Names	GitHub
1	Mohammad Huzaifa	mohammad-huzaifa-627
2	Muhammad Saad	saadthedeveloper
3	Mudassir Khan	muddassir-khan-se

2. Problem Description

Software teams often have more feature ideas than time. Deciding which features to build first is usually based on guesswork. The Kano Model helps by surveying users and classifying each feature as Attractive, One-dimensional, Must-be, Indifferent, Reverse, or Questionable. Doing this by hand from survey data (for example, a Google Forms export) is slow, repetitive, and error-prone, and there is no simple free tool for student and small teams.

3. Objectives and Intended Users

Objectives

· Automate Kano classification of features from survey responses.

· Help teams prioritize features using the classification and satisfaction coefficients.

· Show survey results visually through graphs.

· Provide a simple CLI first, with a GUI planned for a later release.

Intended users

· Student software teams choosing features for projects.

· Product managers and startup teams.

· Anyone who runs a Kano survey and wants fast, consistent results.

ID	Requirements
FR1	The system shall read survey responses from a CSV file exported from Google Forms.
FR2	The system shall classify each respondent's answer pair (functional / dysfunctional question) into A, O, M, I, R, or Q using the standard Kano evaluation table.
FR3	The system shall count the categories for each feature and determine its dominant category (ties shown as combined, e.g. A/O).

FR4	The system shall calculate Better (satisfaction) and Worse (dissatisfaction) coefficients for each feature.
FR5	The system shall prioritize/rank features based on the category and coefficients (general order: Must-be, One-dimensional, Attractive, Indifferent).
FR6	The system shall generate graphs of responses per feature. Initial release: 2 to 3 graph types (e.g. stacked bar chart of category counts, Better-Worse scatter plot, per-feature bar chart).
FR7	The system shall print a results summary table in the terminal.
FR8	The system shall export results (table and graphs) to files (CSV / PNG).
FR9	The system shall be operated through a command-line interface (CLI).
FR10	(Later release) The system shall provide a graphical user interface (GUI).

5. Non-Functional Requirements

·**Usability**: Clear CLI commands and helpful error messages.

·**Reliability**: Detect and report malformed or incomplete CSV files instead of crashing.

·**Performance**: Process a typical survey (up to a few hundred responses) in a few seconds.

·**Portability**: Run on Windows, Linux, and macOS with Python 3.

·**Maintainability**: Keep analysis logic separate from the interface so a GUI can be added later without rewriting the core.

·**Testability**: Core classification logic should be covered by unit tests.

6. Prioritized Feature List (Product Backlog)

Must have

1.CSV loading (FR1)

2.Kano classification (FR2)

3.Dominant category per feature (FR3)

4.CLI interface (FR9)Should have

5. Better/Worse coefficients (FR4)

6. Feature ranking (FR5)

7. Stacked bar chart (FR6)

8. Terminal summary table (FR7) Could have

9. Additional graph types, such as Better-Worse scatter (FR6)

10. Export to CSV / PNG (FR8)

Won't have in v1.0 (planned later)

11. GUI (FR10)

7. Assumptions, Constraints, and Known Limitations

Assumptions

·Survey questions come in pairs (functional and dysfunctional) for each feature, with the standard five answer options.

·Responses are provided as a CSV file from Google Forms with a consistent column layout.

Constraints

·Implemented in Python (pandas and matplotlib or equivalent libraries).

·Development happens in a shared public GitHub repository within the semester timeline.

·CLI only for the first versions.

Known limitations

·Results are only as reliable as the survey sample size and quality.

·Ties between categories are reported as combined categories and need human judgement.

·Initial version supports a fixed CSV layout only.

·Requirements may change as the project evolves (agile approach); this document will be updated accordingly.